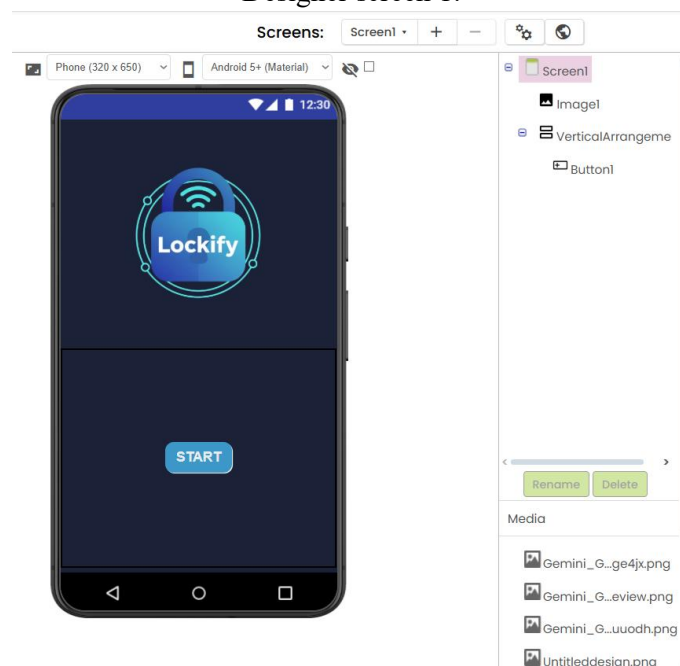


SMART LOCK & SECURITY SYSTEM

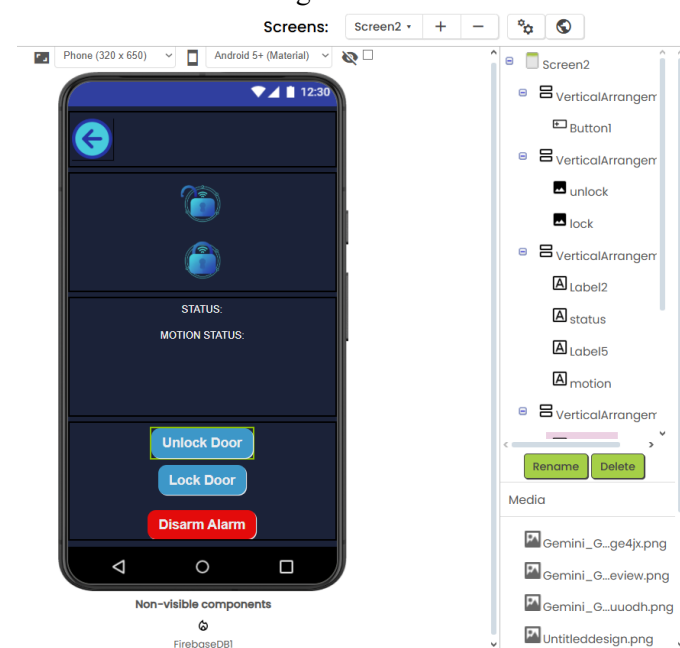
LIST OF EQUIPMENT/SOFTWARE REQUIRED

- Google Firebase
- MIT App Inventor
- Node MCU ESP8266 microcontroller
- Electric buzzer
- PIR Motion Sensor
- Servo Motor
- 5V source
- Jumper wires

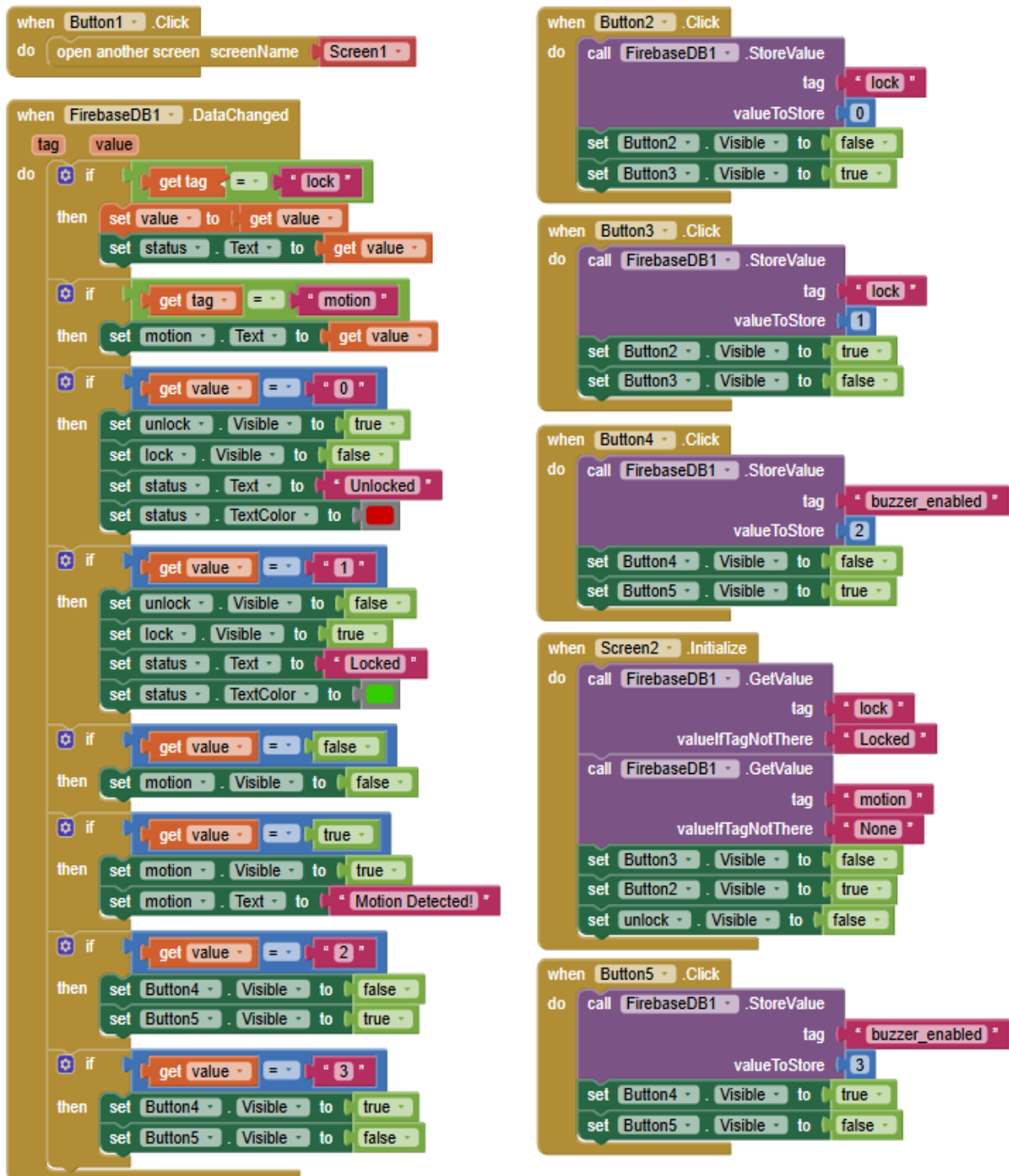
Designer screen 1:



Designer screen 2:

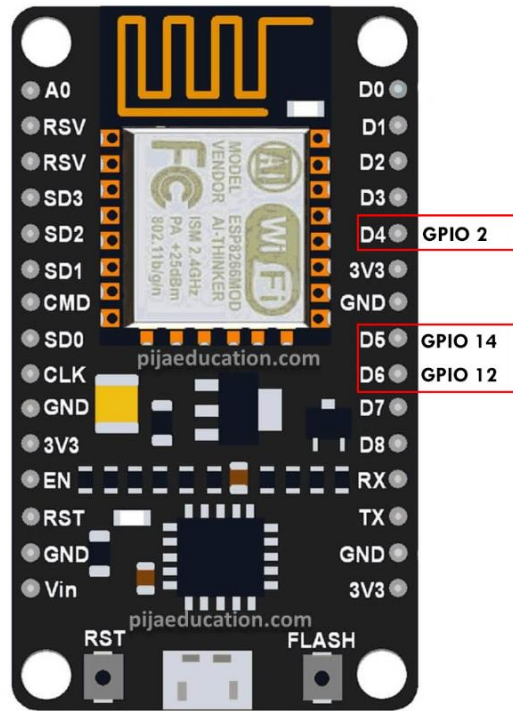


Blocks for Screen 1:



Blocks for Screen 2:

GPIO Mapping with Arduino IDE



Node MCU Pin Connection

Configuration	Connection
SERVO MOTOR (GND)	GND
SERVO MOTOR (POWER)	5V SOURCE
SERVO MOTOR (CONTROL)	GPIO 2
PIR (GND)	GND
PIR (DIGITAL OUT)	GPIO 14
PIR (POWER)	3V SOURCE
BUZZER (+)	GPIO 12
BUZZER (-)	GND

Sample Code:

```
#include <ESP8266WiFi.h>
#include <Firebase_ESP_Client.h>
#include <Servo.h>

#define WIFI_SSID "PisoWIFI"
#define WIFI_PASSWORD "V@leacruz1234"
#define FIREBASE_HOST "lab4-be8c0-default-rtdb.firebaseio.com"
#define FIREBASE_AUTH "eFk7Q2u2gGCSoxLjsLzJDfmOotHgqdxGme1bSJsb"

#define SERVO_PIN D4
#define PIR_PIN D5
#define BUZZER_PIN D6

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
Servo doorServo;
```

```

String currentLockState = "Locked";
unsigned long lastFirebaseCheck = 0;
const unsigned long firebaseCheckInterval = 100;

unsigned long beepStartTime = 0;
bool isBeeping = false;
bool motionDetected = false;
bool lastPirState = false;
bool buzzerEnabled = true;

unsigned long lastManualCheck = 0;
const unsigned long MANUAL_CHECK_INTERVAL = 300;
String lastBuzzerState = "1";

bool needsFirebaseUpdate = false;
bool newMotionState = false;
unsigned long lastFirebaseTime = 0;
const unsigned long FIREBASE_COOLDOWN = 100;

void setup() {
  Serial.begin(115200);
  delay(1000);

  doorServo.attach(SERVO_PIN);
  doorServo.write(0);
  delay(500);

  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("\n\n=== SMART LOCK & SECURITY SYSTEM ===");
  Serial.println("Initializing...");

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  int wifiAttempts = 0;
  while (WiFi.status() != WL_CONNECTED && wifiAttempts < 20) {
    delay(500);
    Serial.print(".");
    wifiAttempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi Connected!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("\nWiFi Connection Failed! Rebooting...");
  }
}

```

```

    delay(2000);
    ESP.restart();
}

config.database_url = FIREBASE_HOST;
config.signer.tokens.legacy_token = FIREBASE_AUTH;
Firebase.begin(&config, &auth);
Firebase.reconnectWiFi(true);

if (Firebase.ready()) {
    Firebase.RTDB.setString(&fbdo, "/lock", "Locked");
    Firebase.RTDB.setString(&fbdo, "/buzzer_enabled", "1");
    Firebase.RTDB.setBool(&fbdo, "/motion", false);
    Serial.println("Firebase initialized successfully!");
}

Serial.println("=== SYSTEM READY ===");
Serial.println("Servo and Motion Detection Active!");
}

void loop() {
    unsigned long currentTime = millis();

    if (currentTime - lastFirebaseCheck >= firebaseCheckInterval) {
        lastFirebaseCheck = currentTime;

        if (Firebase.RTDB.getString(&fbdo, "/lock")) {
            String lockCommand = fbdo.stringData();

            lockCommand.replace("\"", "");
            lockCommand.trim();

            if (lockCommand == "0" && currentLockState != "Unlocked") {
                Serial.println("🔓 Unlock Command Received - Moving Servo!");
                doorServo.write(180);
                currentLockState = "Unlocked";
                Firebase.RTDB.setString(&fbdo, "/lock", "Unlocked");
                Serial.println("✅ Door Unlocked!");
            }
            else if (lockCommand == "1" && currentLockState != "Locked") {
                Serial.println("🔒 Lock Command Received - Moving Servo!");
                doorServo.write(0);
                currentLockState = "Locked";
                Firebase.RTDB.setString(&fbdo, "/lock", "Locked");
                Serial.println("✅ Door Locked!");
            }
        } else {
            Serial.println("Failed to read lock from Firebase");
        }
    }
}

```

```

bool currentPirState = digitalRead(PIR_PIN);

if (currentPirState && !lastPirState) {
    motionDetected = true;
    Serial.println("👁️ MOTION DETECTED - INSTANT!");

    needsFirebaseUpdate = true;
    newMotionState = true;

    if (buzzerEnabled && !isBeeping) {
        isBeeping = true;
        beepStartTime = millis();
        digitalWrite(BUZZER_PIN, HIGH);
        Serial.println("🔊 BUZZER ON - INSTANT");
    }
}

if (!currentPirState && lastPirState && motionDetected) {
    motionDetected = false;
    Serial.println("👋 Motion ended - INSTANT");

    needsFirebaseUpdate = true;
    newMotionState = false;
}

lastPirState = currentPirState;

checkManualControl();

handleFirebaseUpdate();

if (isBeeping && (millis() - beepStartTime >= 10000)) {
    isBeeping = false;
    digitalWrite(BUZZER_PIN, LOW);
    Serial.println("🔊 Beeping stopped (10 sec)");
}

delay(50);
}

void checkManualControl() {
    unsigned long currentTime = millis();

    if (currentTime - lastManualCheck >= MANUAL_CHECK_INTERVAL) {
        lastManualCheck = currentTime;

        if (Firebase.RTDB.getString(&fbdo, "/buzzer_enabled")) {
            String newState = fbdo.stringData();

            if (newState != lastBuzzerState) {
                lastBuzzerState = newState;
            }
        }
    }
}

```

```

    if (newState == "2") {
        buzzerEnabled = false;
        digitalWrite(BUZZER_PIN, LOW);
        isBeeping = false;
        Serial.println("🔊 BUZZER DISABLED - INSTANT POLLING");
    } else if (newState == "3") {
        buzzerEnabled = true;
        Serial.println("🔊 BUZZER ENABLED - INSTANT POLLING");

        if (motionDetected && !isBeeping) {
            isBeeping = true;
            beepStartTime = millis();
            digitalWrite(BUZZER_PIN, HIGH);
        }
    }
}
}
}

void handleFirebaseUpdate() {
    unsigned long currentTime = millis();

    if (needsFirebaseUpdate && (currentTime - lastFirebaseTime >= FIREBASE_COOLDOWN)) {
        if (Firebase.ready()) {
            Firebase.RTDB.setBool(&fbdo, "/motion", newMotionState);
            Serial.println("🔊 Firebase: " + String(newMotionState ? "MOTION" : "NO MOTION"));
        }

        needsFirebaseUpdate = false;
        lastFirebaseTime = currentTime;
    }
}
}

```

Output:

